README for ESP32 live Temperature and Humidity dashboard (simulated thermal management system)

README for ESP32 live Temperature and Humidity dashboard (simulated thermal management system) 1
Project overview 1
Aim of the project / success 2
Temperature threshold table 2
ESP32 System diagram 3
Python System diagram 3
High level system level diagram 3
Hardware list & circuit diagram 4
Physical circuit labelled diagram: 5
Circuit Schematic 5
ESP32 6
Networking – data flow 6
Python client, data visualisation 7
Problems encountered and the decisions to overcome them 7
Limitations and future improvements 8
Software – Python code and ESP32 code 8
Python Figure 9
Setup & instructions 10

Project overview

- put simply, my project measures the temperature and humidity of the environment using a DHT11 sensor. Based on the temperature it powers a 6v DC motor with a certain duty cycle to simulate cooling (as a fan). This is through a BJT 2N2222A transistor

switching circuit with a power supply. On the breadboard there are also indicators for the temperature states and motor speed states via colour-coded LEDS, an Adafruit OLED screen to display the current temperature, humidity and motor speed. It transmits this data over a local network to a computer client running a python script where the data is parsed and then plotted.

Aim of the project / success

- the actual aim of the project was to simulate a thermal management system measuring temperature and humidity. Then control a DC motor using PWM mimicking a cooling fan, and then visually see the motor speed, temperature and humidity change with time using a live python script over my Wi-Fi network using an ESP32.

  For the project to be considered successful I simply wanted it to be able to do the following:
- read the temperature and humidity from the DHT11 sensor
- display said readings on a small OLED panel, as well as the motor speed
- indicate the temperatures visually on the breadboard with LEDS
- adjust motor PWM duty cycle based on temperature
- send data to a python client via HTTP /data endpoint
- log data in python and plot it
- save data via writing data in csv form to a raw txt file

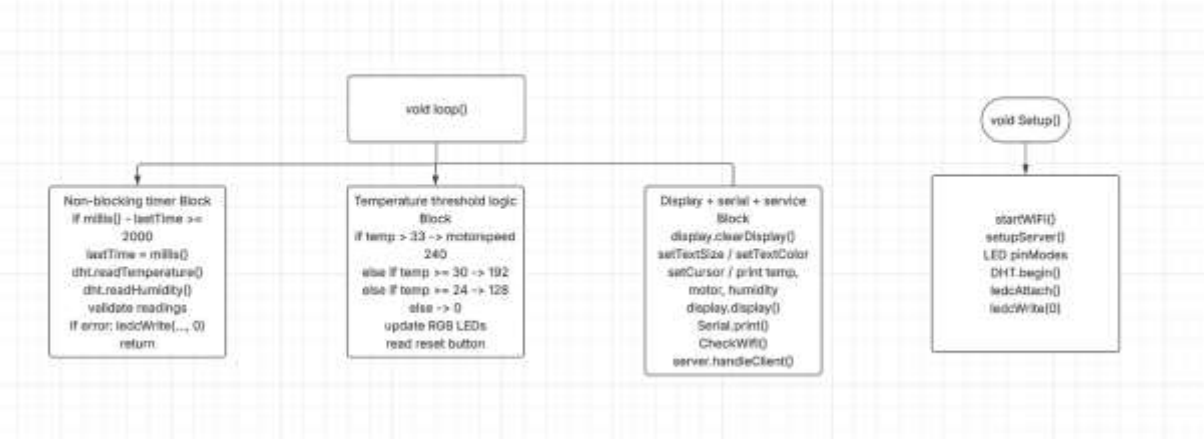
Temperature threshold table

- I mentioned motor speed is controlled by temperature. I did this with simple if statements checking if the temperature was above a certain threshold within the main loop. See the table below:

Temperature range	LED state	Motor PWM value	Purpose?
Below 24c	Blue	0/255 0%	The environment is cool, no need for a fan
24-29.9c	Green	128/255 Almost 50%	The temperature is increasing but still not high, so moderate cooling
30-33c	amber	192/255 Almost 75%	Temperature is now high, increased cooling
>33c	red	240/255 >90%	Very high temperature, almost maximum cooling.

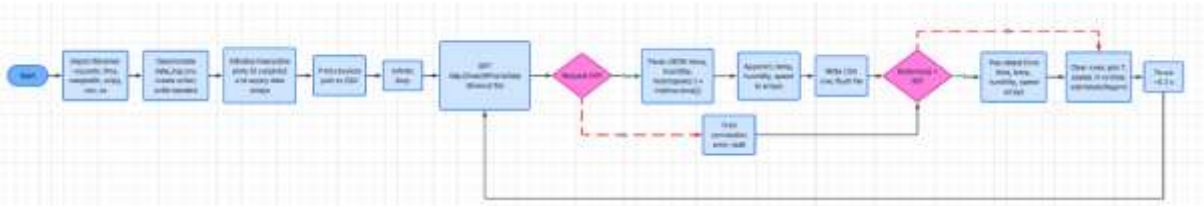
- The temperature ranges that I created were not necessarily some accurate real world temperature ranges for a given cooling system. Instead, I made the thresholds such that in my room I could easily push the temperature above and bellow every threshold for testing purposes. Since my room is almost always around 24-27 degrees, I choose the values you see above.
- As you can see the motor PWM values are some portions of 255. As a result, there are percentages for each PWM value.

ESP32 System diagram



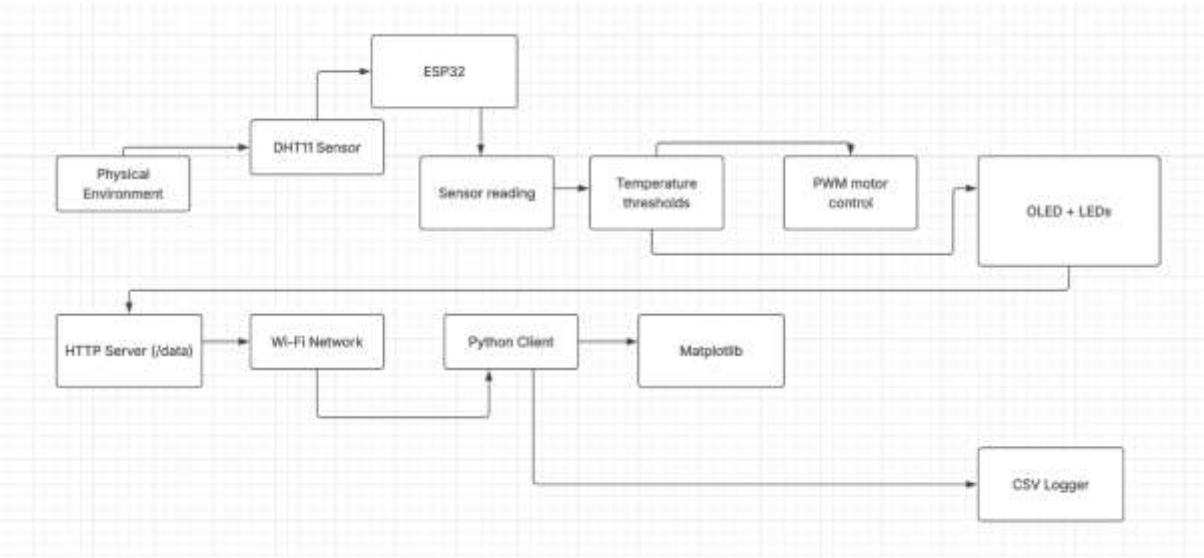
- Do note that for the diagram above it is very high level. The blocks do not “call” each other or directly cause the other, they are separate categorised parts of the main loop. Ive split them for simplicity purposes.

Python System diagram



High level system level diagram

- Here is the diagram that shows the relationship between the different systems of the full project. Again, this is really abstracted, and arrows do not necessarily mean they cause the next thing to happen. Rather this is the sequence / general structure of what is happening when everything has been initialized.



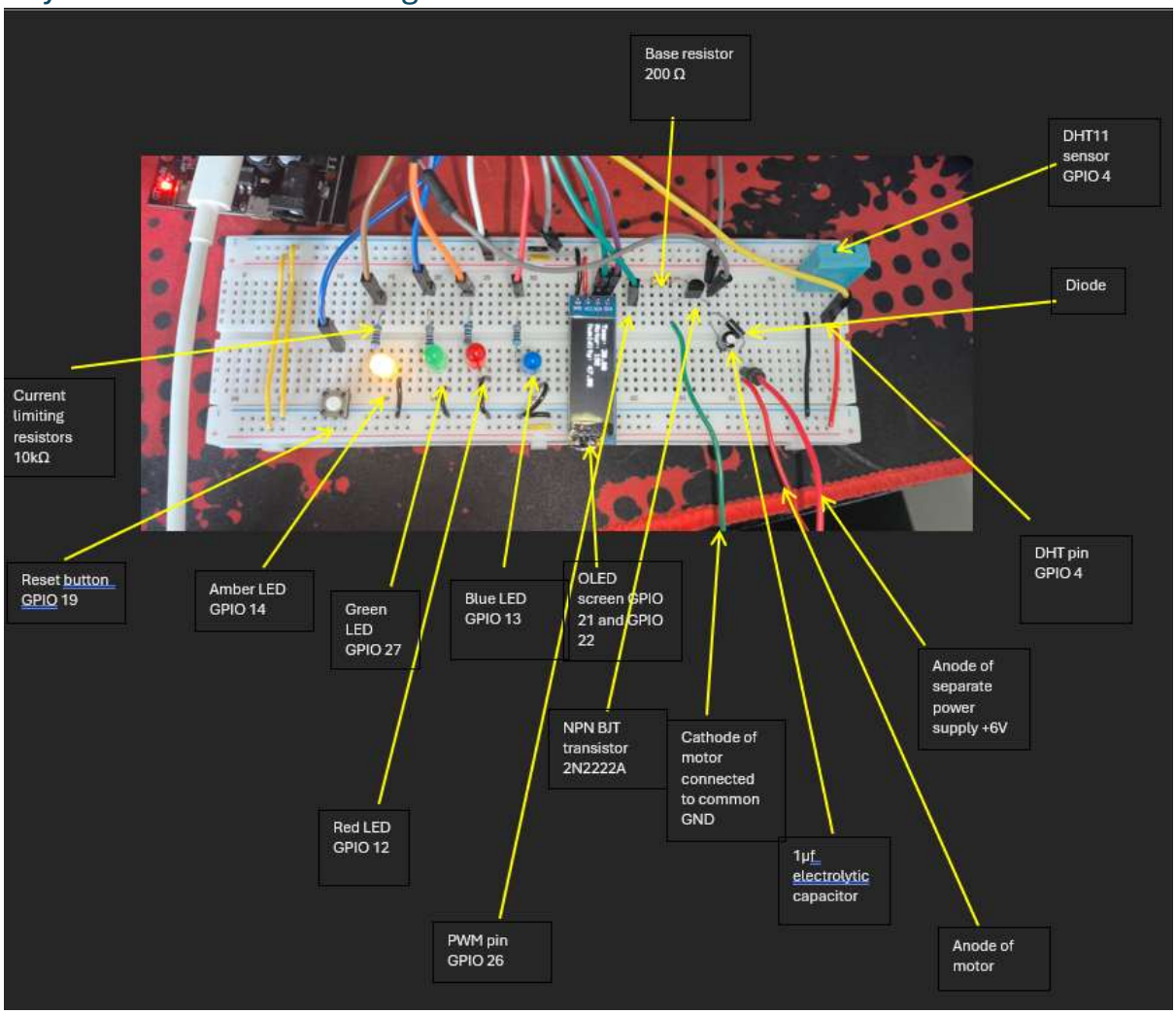
Hardware list & circuit diagram

Several components were used for this project. As well as equipment. They are as follows:

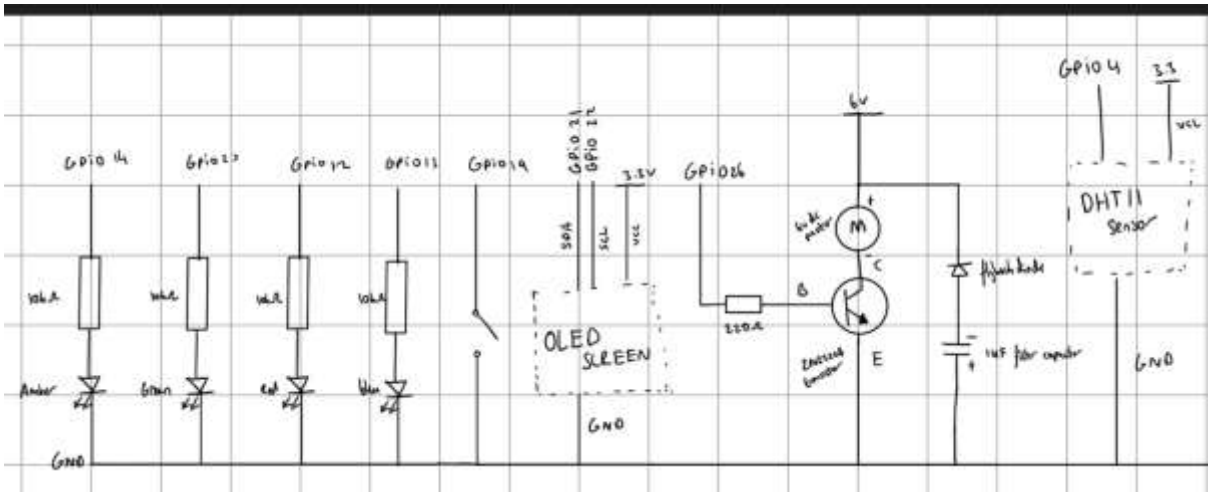
- DHT11 sensor
- esp32 dev board
- breadboard
- wires
- buttons
- various colour LEDS
- 6v 300rpm dc motor
- diode
- capacitor
- resistor
- Adafruit OLED screen
- NPN BJT 2N2222A transistor
- external 30V 5A power supply

I had these components laying around, so this system was designed around these components. There are limitations with each component which I will discuss later.

Physical circuit labelled diagram:



Circuit Schematic



ESP32

- the esp32 is the brain of this project. I choose this microcontroller because I knew I wanted to transmit data wirelessly and hence it was perfect as it has Wi-Fi. The esp32 interfaces with the DHT11 sensor to measure and store the temperature and humidity. I used a non-blocking time interval with millis() to make sure measurements had suitable timing. The esp32 generally manages GPIO for all components on the board e.g. LEDS, the OLED display, the button, and the DHT11 sensor itself. It has two main loops as well as several function definitions. These are setup() and loop(). Setup initialises the pin mode for the components as well as manages the initialisation of Wi-Fi (establishing a connection). Loop contains the threshold logic for motor PWM as well as the led indicators. It checks the temperature, mapping the temperature to its corresponding LED and then adjusts the PWM of the pin connected to the motor circuit. I also use the esp32 to use internal PWM capability via ledc. The esp32 itself could not power a 6v motor which is all I have and therefore I had to use the esp32s PWM signal as a control signal in combination with the transistor and an external supply to rapidly switch the motor. I also use the esp32 to allow I2C communication between the esp32 and the OLED screen. This displays each of the measurements and motor speed. And finally, the esp32 initialises the web server whereby we transmit the data wirelessly. This is on port 80. The esp32 stores the temperature data inside its SRAM and then when a client requests the IP assigned to the esp32 and /data the esp32 will convert the data to Json and transmit the data via a HTTP 200 OK response.

Networking – data flow

- in terms of networking. The entire project relies on a client / server relationship. A very simple ascii demonstration would be:

python client ➔ HTTP GET request ➔ ESP32 web server ➔ JSON data ➔ MATPLOTLIB plotting ➔ exporting csv data

- initially, the esp32 run a HTTP server on port 80 via “WebServer server(80);”. I define the URL path as the IP of the esp32 assigned by the local network followed by /data. I wanted this to be the simplest resource rather than a HTML web page. Though, in the future this could be made into another dashboard with CSS and JavaScript.
- After this, the python script automatically asks for data via HTTP GET requests via the request’s library. It targets the ESP32s Ip address e.g. 000.000.0.000/data.
- When the ESP32 gets thee request it checks its SRAM, formats the temperature, humidity and motor speed data into a JSON string and then sends this back to the client with a HTTP 200 OK status header. When the python client gets this back, the JSON will be converted into a python dictionary via .json(). This individually gets the temp, humidity, and motor speed variables. This is added to their respective arrays.

Python client, data visualisation

- In my python script I have an independent time system. The time is not recorded via millis() as it is inside of the esp32, but rather through python's time function as seen by int(time.time()). This makes timing more consistent as this time function simply counts in seconds on the horizontal axis. This makes the dashboard more linear. It is also more robust because when the esp32 transmits delayed measurements a consistent timeframe will still be used. As for plotting I used a fixed maximum of 30 data points on any of the three graphs at a given moment tin time. I used .pop(0) to remove the oldest index across all four data arrays. This massively improved performance and works well with the fact that I transmit with csv. You can still see the live data through this graph, but old measurements are not lost either. The csv file reading is simply done via first creating a new file with open("data_log.csv", "w"). after this I create column titles and then inside of the loop I take the different data variables and write them into a row. This would be in the format "1719876541,25,60,50". And finally, I use file.flush(). This physically stores the data on my computers SSD. Rather than discarding it instantly.

Problems encountered and the decisions to overcome them

I encountered many issues. Most of these were during development and I did not think of beforehand. They are as follows:

- Motor stalling: at various duty cycles, e.g. 40%, 50%, 60% the motor would not spin. However, using a multimeter and checking the display of my power supply I could see that current was being drawn. Despite this, if I manually moved the motor shaft with my finger only just slightly or even disconnected and connected the supply again it would start spinning again. I therefore knew this was just a startup issue. When the motor started spinning it would work as intended. I realised that the cause for this was down to overcoming the initial friction within the motor.
- I used a breadboard for this project. This is not inherently bad for a prototype. But I found that the rails had a resistance of 0.3 ohms. Due to the large amount of electrical noise from the motor due to the switching voltage there was actually small voltage drops across the rails. Which meant the GND rail (as I had initially wired all grounds together to form one large common ground row) was floating slightly. This meant that the circuit would not function properly and as a result the motor would not spin at all. I fixed this by simply moving the transistors circuit GND directly to a separate GND pin on the ESP32. That is the node that is connected to the transistor's emitter. This worked perfectly, I effectively bypassed all the noise. After I had done this the motor worked perfectly.
- I was not sure on what transistor to use so I tried n type and p type MOSFETS. Neither of these worked, I soon realised this was because they required quite a large voltage to

conduct (> 3.3v) so I switched this for a BJT. The base current requirement was achievable.

- Motors are inductive loads and require quite a lot of current to get going. So, I needed a power supply. This was part of the design to use the ESP32 to send control signals to the transistor to rapidly turn the motor on and off. As mentioned before, this did require a common ground between both circuits.
- The motor was slightly inconsistent with movement due to the lack of a decoupling capacitor and a flyback diode. I was watching videos online and saw how sudden changes in current can damage microcontrollers like the ESP32 so immediately added them.
- I found it hard to choose a value for the PWM frequency myself, so I searched common frequency used for hobbyist motors and found that 5khz was a common value.
- I was initially using blocking delays. But improved this using millis(), a non blocking timer. This was a massive improvement because the ESP32 was dead for those 2 seconds. Not responding to any network requests etc.
- I was encountering several issues with JSON formatting, the python client was anot accepting the strings due to syntax errors which I was not aware of.

Limitations and future improvements

- First and foremost, the DHT11 is a very basic sensor. It doesn't have the best accuracy (data sheet shows accuracy can range by up to 2 degrees)
- As well as this, it has a slow update time compared to other sensors like a DHT22 or SHT31. Though since I bought this component well in advance it is the best I have.
- The motor control is very very simple. There are simply fixed temperature ranges which it responds to as opposed to an adjustable dynamic algorithm. I could change this by making the temperature change cause proportional motor speed increases/decreases. This would include smoothing the output.
- The current communication aspect of the project is functional but again, could be improved. It is easy to debug however there is quite a lot of delays and requests from python → ESP32 can time out often. I could improve this by moving my ESP32 closer to the router or use a different communication method which is made for constant updates like MQTT.
- The motor moving does not cause changes in temperature. This is not a fully functioning closed loop control system. In the future I could adjust this or scale it so that this is possible.
- There is no encryption or security. Since this is an IoT project which uses a HTTP server and network it should have some sort of authentication. I may add this in the future through authentication tokens. Or use HTTPS.

Software – Python code and ESP32 code

- I will include the full source code for the project in the repository. Though here is a very brief explanation of the code as a whole for both the ESP32 code and the python code

respectively. Do note that inside of the source code there are a lot of comments which you can read to better understand it.

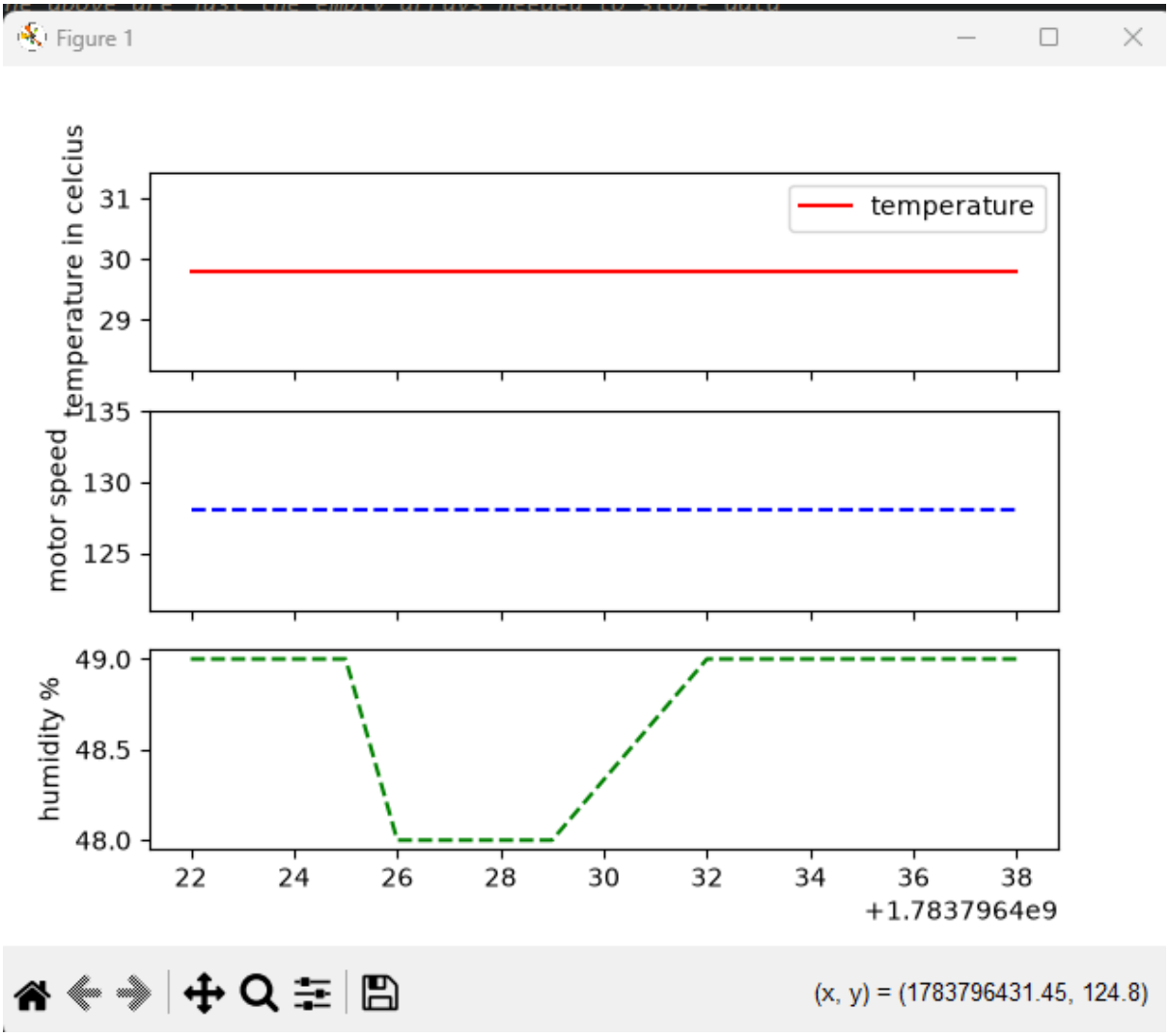
ESP32:

- This is the main code used to power the ESP32. It is a singular file which contains all of the logic that powers the breadboard. E.g. GPIO definitions, initialising the sensors the OLED display controlling motor PWM. The temperature threshold logic, as well as establishing the web server and handling data.

Python:

- This is a very simple and short script which is ran by a computer client. It constantly sends HTTP get requests to the ESP32 /data endpoint. When it receives the JSON data it stores each separate data type in an array and then plots them all against time using MATPLOTLIB. Finally, it writes the data to a CSV file.

Python Figure



Setup & instructions

- Build the circuit according to the schematic and labelled diagram in this repository. (You may change GPIO pins if needed — just ensure the code matches your wiring.)
- Power the motor using a 6 V supply with a ~600 mA current limit.
- Flash the ESP32 with the provided .ino file and reset the board.
- Run the Python client (.py file) in your preferred IDE (e.g., VS Code).
- Once both programs are running, the Python dashboard will begin plotting live temperature, humidity, and motor speed in real time.
- Use your own Wi-Fi network SSID and password — these are not included in the code.

Troubleshooting

- Make sure all grounds are properly connected. If the ESP32 throws errors when motor spins move the transistors emitter ground to a dedicated GND pin on the ESP32
- If motor does not spin at certain PWMs, twist the shaft of the motor. If this does not work press the reset button and release.
- Use your own WIFI password and SSID.
- If python script times out it is mainly Wi-Fi signal strength issue or ESP32 Wi-Fi connection issues.
- For motor terminals curl the wire tip and hook it properly. Loose wires can cause random jumps in motor speed and spikes in current etc.
- Double check DHT11 pinout with datasheet.
- Do the same for the 2N222A
- Avoid long wires and old breadboards due to electrical noise from the motor as well as unreliable connections.